

PROGRAMMAZIONE I

Laboratorio Turno C
FAB (Informatica Musicale)

Docente: MARCO BRESSAN

Laboratorio di Programmazione I

Chi: turno C, cioè FAB (Informatica Musicale)

Dove e quando: qui, ogni mercoledì

Come: con un PC e GNU/Linux (se non ce l'avete, installatelo)

Perché: programmare si impara facendo

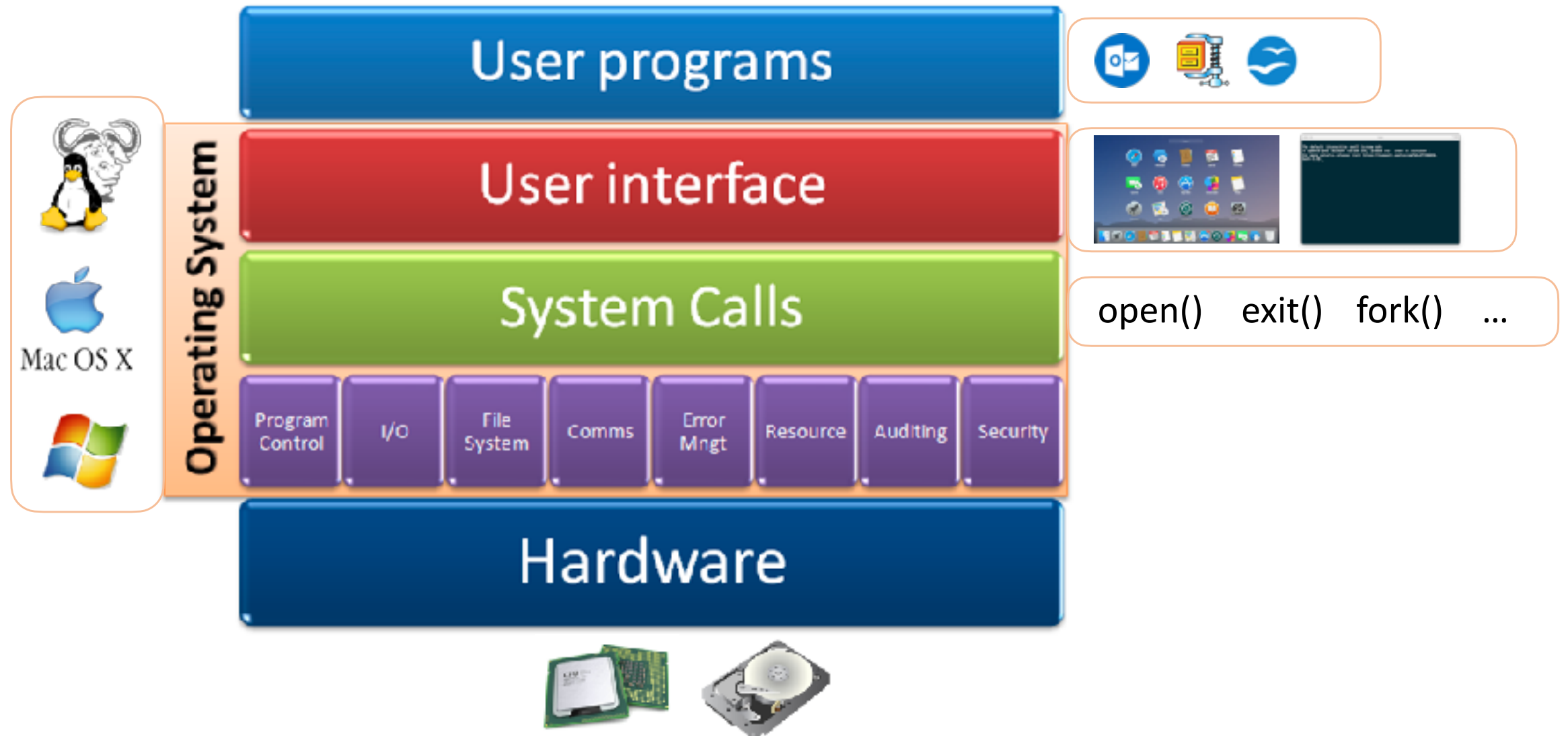
Oggi

1. Com'è fatto un computer

2. La shell

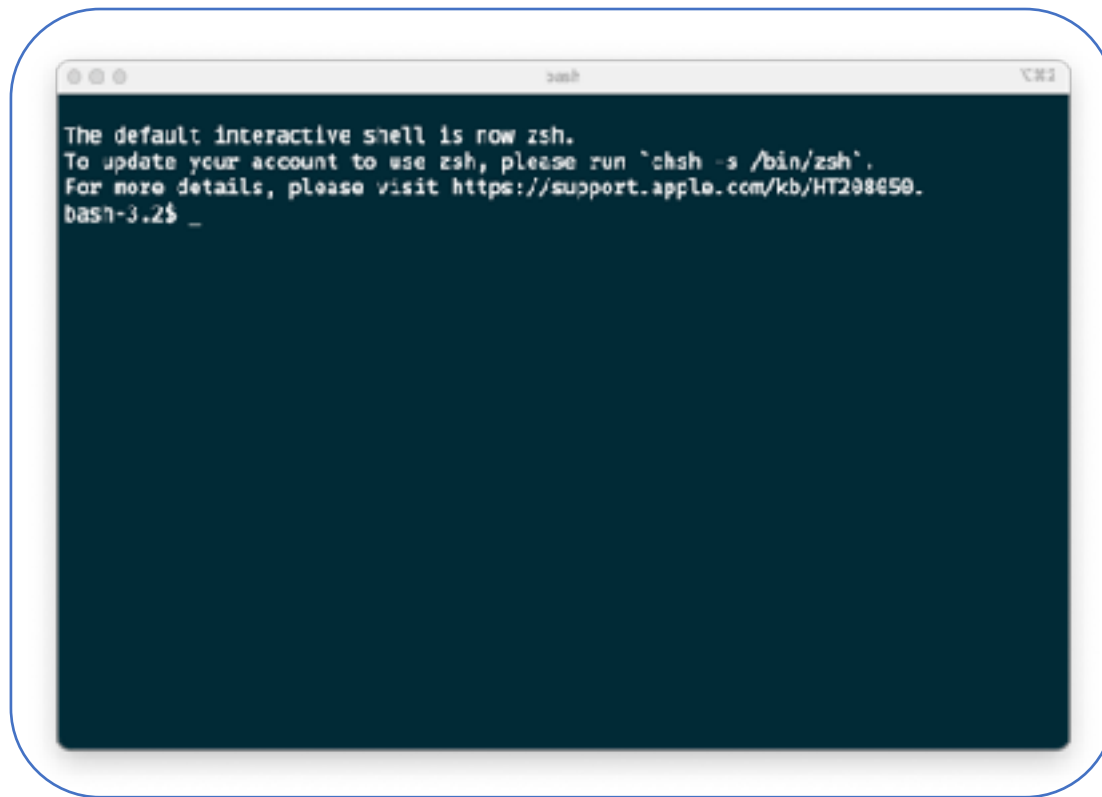
3. Scrivere un programmino

Com'è fatto un computer

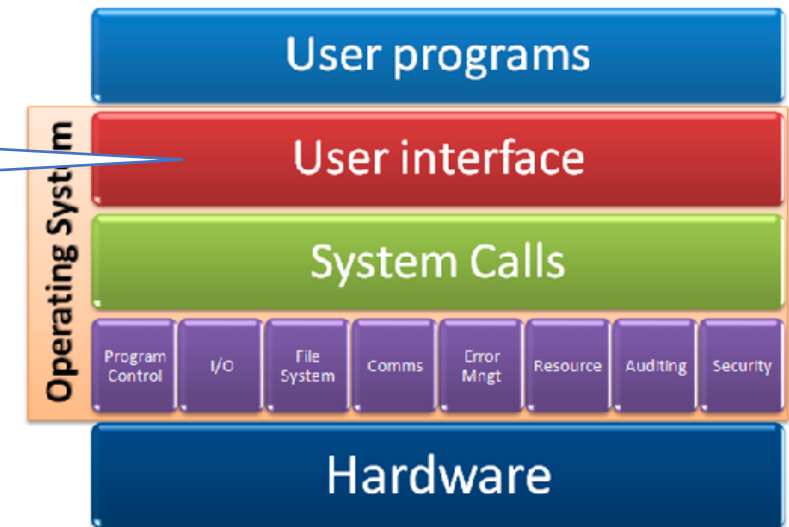


La shell

Interfaccia **testuale** del s.o.



```
zsh
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208450.
bash-3.2$ _
```



La shell

Ne esistono molte:

sh, zsh, bash, tcsh, ...

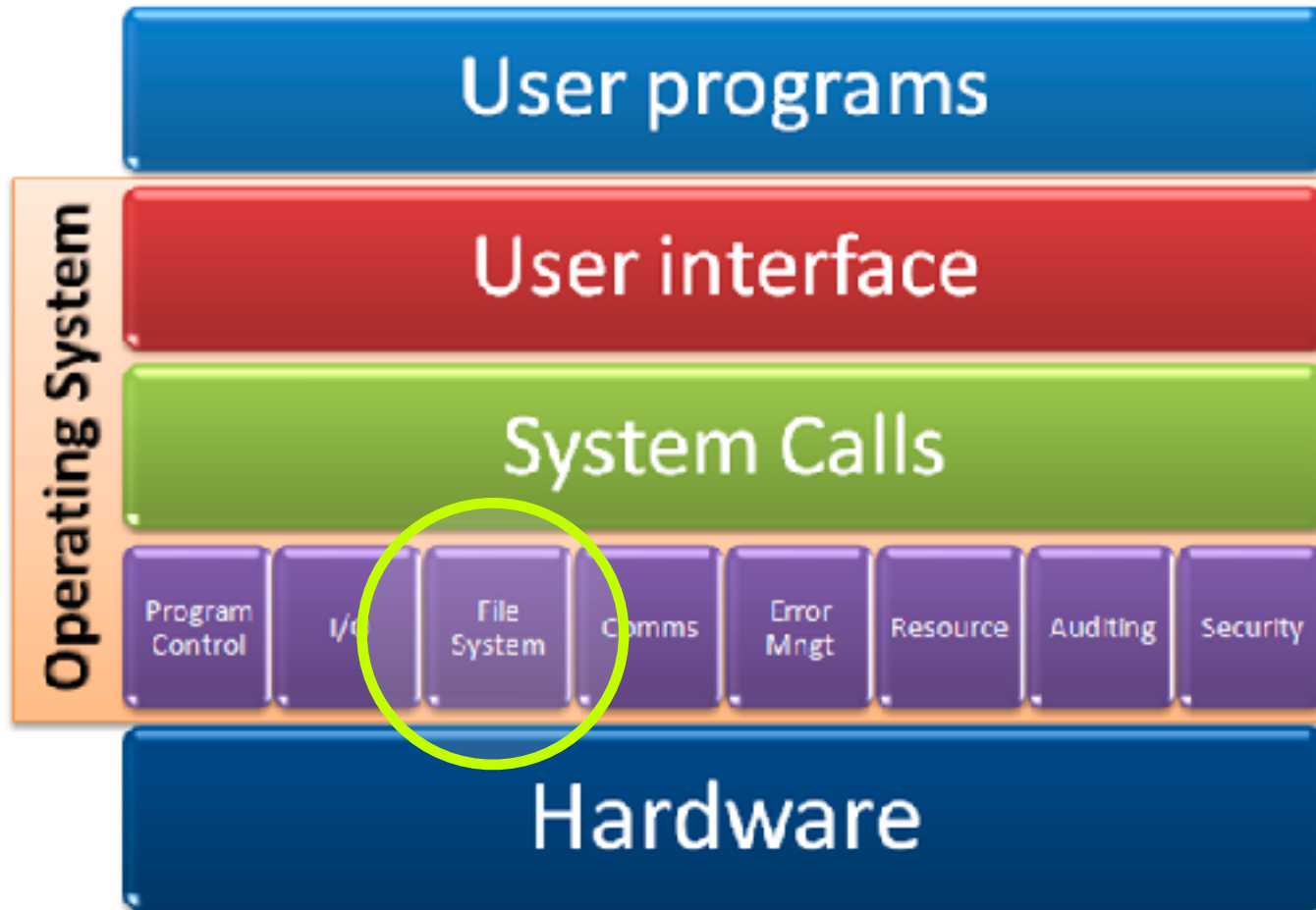
```
bash
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
bash-3.2$ _
```

```

4  +-----+ 1 nateded lease 500 Jan 31 2002 H2Cdees
4  +-----+ 1 nateded lease 4004 Mar 4 11:00 M11-A Grid.ppt
958 +-----+ 1 nateded lease 955198 Oct 4 18:05 NEUFALGrid.z1:
258 +-----+ 1 nateded lease 256816 Mar 22 2005 Nannoung2.doc
12904 +-----+ 1 nateded lease 1252708 Oct 1 2001 NetBeans-release321.Lan.gc
4  d-----+ 2 nateded lease 4096 Sep 13 15:41 Nuova controll:
22 +-----+ 2 nateded lease 25084 Nov 21 2001 UMEs -5th thesis.doc
4  d-----+ 6 nateded lease 4096 May 13 17:14 OFFETEIA
46 +-----+ 1 nateded lease 22746 Sep 27 00:15 Hran-Hi Grid.xls
76 +-----+ 1 nateded lease 73980 Jul 24 17:57 OneCarlo.qnumeric
40 +-----+ 1 nateded lease 28400 Jun 20 2001 OneCarlo.xls
4  +-----+ 2 nateded lease 2624 May 29 2001 OneCarlo.xls, numeri...
38 +-----+ 1 nateded lease 37916 Jul 15 08:55 P15E233IE.png
404 +-----+ 2 nateded lease 429266 Jan 22 2001 PAC_med.Latin_.pdf
276 +-----+ 1 nateded lease 274623 Jun 22 2001 PAC med:titating on enotions
116 +-----+ 1 nateded lease 13146 Nov 27 2001 PAC zip
4  d-----+ 8 nateded lease 4096 Mar 26 2002 PHETA
32 +-----+ 1 nateded lease 32700 Dec 10 2001 PHOTOGRA HATCH FOR THE HTP IFS IN herbold-I issues.doc
32 +-----+ 1 nateded lease 32744 Oct 30 2001 Pech.doc
354 +-----+ 1 nateded lease 357220 Aug 29 2001 Popc-1.jpg
144 +-----+ 2 nateded lease 129746 Sep 5 16:54 PosterTemplate.doc
1856 +-----+ 1 nateded lease 1875745 Apr 10 2001 Preellashno
246 +-----+ 1 nateded lease 24244 Apr 10 2001 FreshFish.zip

```

II file system



Il file system

Gestisce organizzazione delle informazioni su disco (HDD, chiave USB, ...)

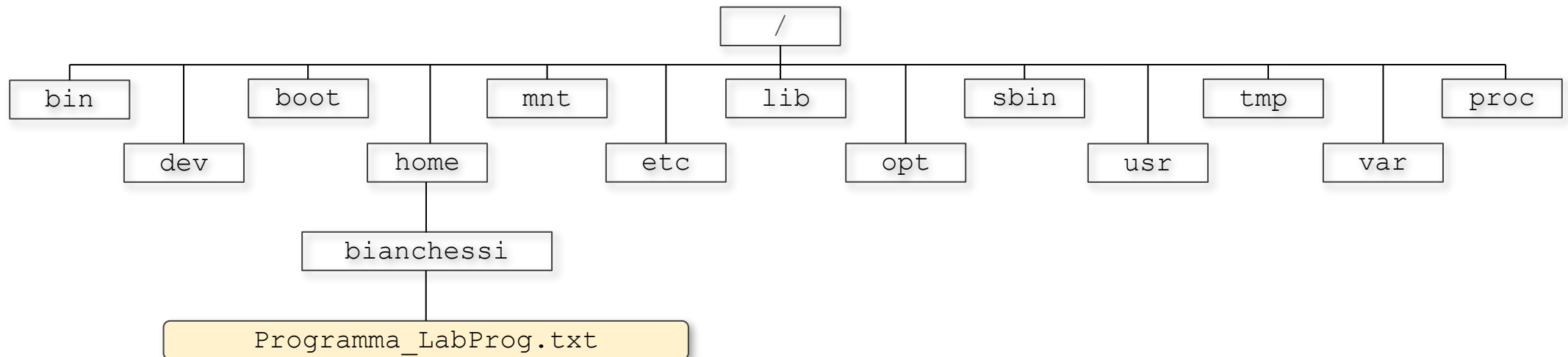
Concetti chiave:

- **file**: contiene una singola “unità logica” di informazione
file di testo (README.txt); video musicale (concerto.mp4); eseguibile (word.exe); ...
- **directory** (o cartella o folder): “contenitore” di altri file e/o directory
C:\\Utenti\\Marco\\Downloads; /var/local/bin/go

Il file system in GNU/Linux

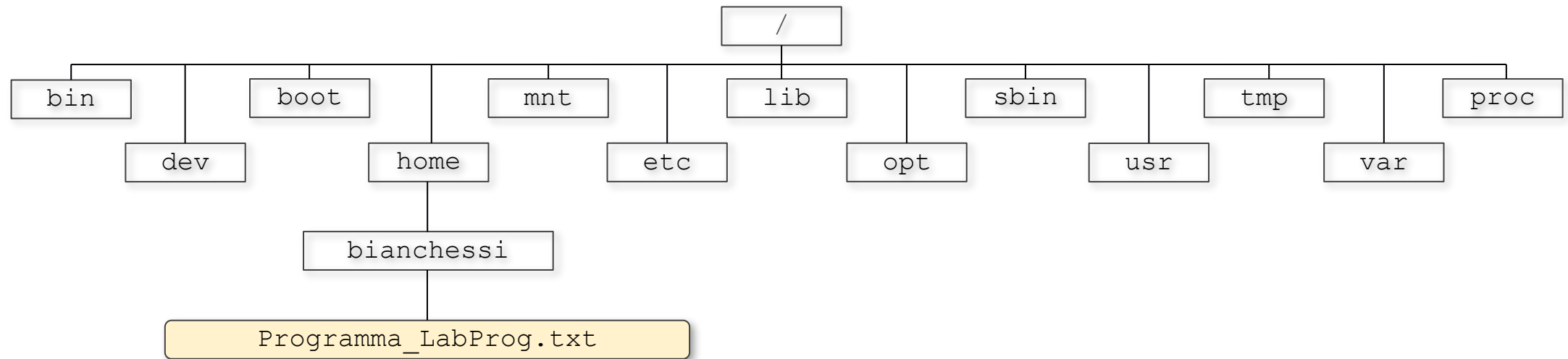
Struttura **gerarchica** ad albero rovesciato

- alla base si trova la **directory principale**, detta *radice (root)*
- le **directory intermedie** costituiscono i **nodi** dell'albero
- i **file** (insieme alle eventuali directory vuote) rappresentano le **foglie**



Il file system in GNU/Linux

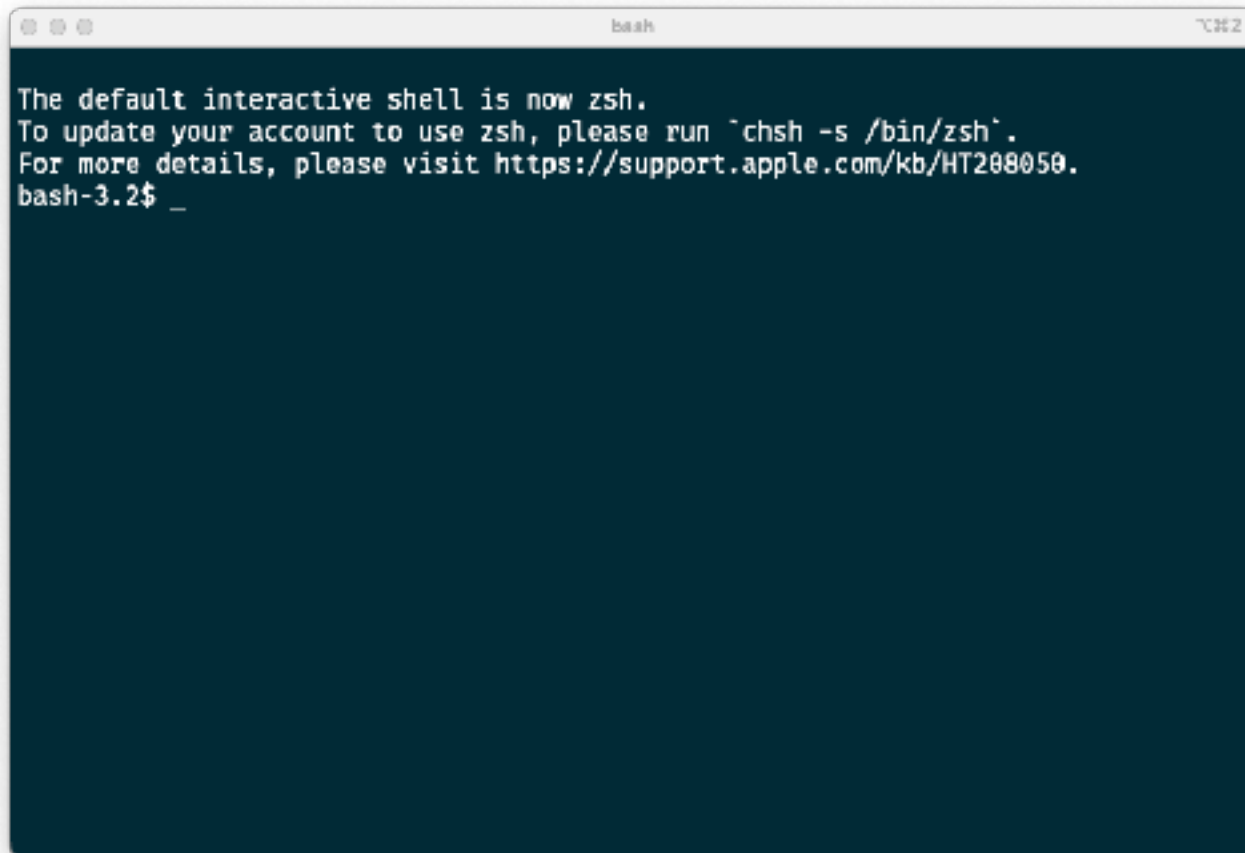
Ogni file è individuato **univocamente** dalla sua posizione nell'albero, cioè dal cammino (**pathname** o **path**) che parte dalla radice e giunge al file.



`/home/bianchessi/Programma_LabProg.txt`

Il file system in GNU/Linux

Ora apriamo un **terminale** e giochiamo con file system e alcuni comandi basilari:

A terminal window with a dark blue background and white text. The window title bar shows 'bash' and a window icon. The text inside the terminal reads: 'The default interactive shell is now zsh. To update your account to use zsh, please run `chsh -s /bin/zsh`. For more details, please visit https://support.apple.com/kb/HT208050. bash-3.2\$ _'.

```
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
bash-3.2$ _
```

pwd
cd
mkdir
ls
cp
mv
cat
...

pwd (print working directory)

directory corrente: il punto del filesystem in cui la shell “*si trova*” attualmente.

Memorizzata nella variabile PWD, stampata dal comando pwd.

Quando si avvia una shell, la directory corrente è la **home directory** dell'utente.

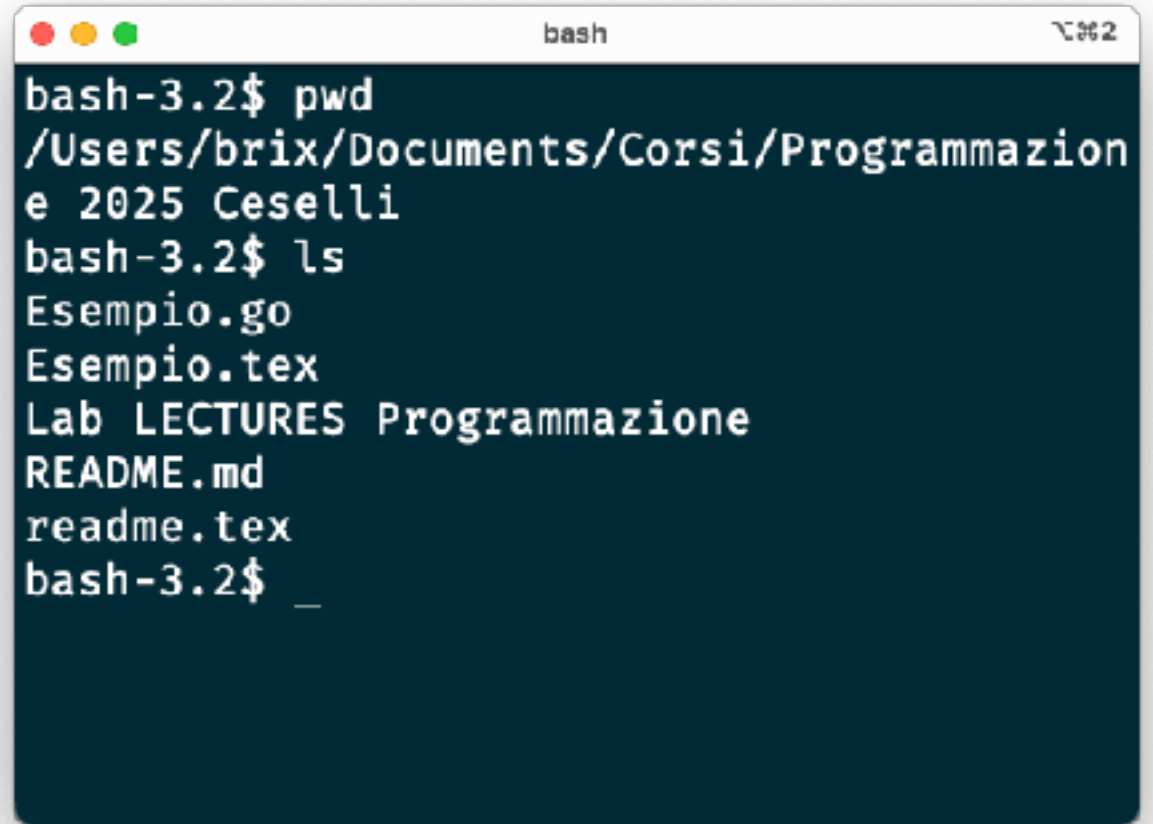
(MAIUSCOLO e minuscolo sono **diversi** in GNU/Linux).

A terminal window titled 'bash' with standard macOS window controls (red, yellow, green buttons) in the top-left corner. The terminal shows a sequence of commands and their outputs: 'bash-3.2\$ pwd' followed by '/Users/brix', 'bash-3.2\$ echo \$PWD' followed by '/Users/brix', and 'bash-3.2\$ _' followed by a blank line. The text is white on a dark blue background.

```
bash-3.2$ pwd
/Users/brix
bash-3.2$ echo $PWD
/Users/brix
bash-3.2$ _
```

ls (list directory contents)

Elenca il contenuto della directory specificata. Di default è la directory corrente

A terminal window titled 'bash' with a dark blue background and white text. The window shows the execution of the 'pwd' and 'ls' commands. The 'pwd' command outputs the current directory path: '/Users/brix/Documents/Corsi/Programmazione 2025 Ceselli'. The 'ls' command lists the contents of the current directory: 'Esempio.go', 'Esempio.tex', 'Lab LECTURES Programmazione', 'README.md', and 'readme.tex'. The prompt 'bash-3.2\$' is visible at the start of each line, and a cursor is shown at the end of the last line.

```
bash-3.2$ pwd
/Users/brix/Documents/Corsi/Programmazione
e 2025 Ceselli
bash-3.2$ ls
Esempio.go
Esempio.tex
Lab LECTURES Programmazione
README.md
readme.tex
bash-3.2$ _
```

cd (change directory)

- Cambia la directory corrente.
- Comportamenti:
 - Senza argomenti → ci si sposta nella home directory dell'utente.
 - Con un pathname come argomento → ci si posiziona nella directory indicata.

```
cd
# si entra nella home directory
cd /home/utente/Documenti
# ci si sposta nella cartella Documenti
```

Pathname di file e directory

Un pathname può essere indicato in due modi:

- **Assoluto:** l'intero percorso a partire dalla root directory (/).

`/home/utente/Documenti/file.txt`

- **Relativo:** il percorso a partire dalla directory corrente.

`Documenti/file.txt`

Abbreviazioni:

il singolo punto `.` indica la directory corrente

il doppio punto `..` indica la directory genitore

Uso di . e .. con il comando cd

```
cd .    # rimane nella stessa directory
```

```
cd ..   # si sposta nella directory che contiene  
        # quella corrente
```

```
cd ../Documenti    # dalla directory padre entra in  
                   # Documenti
```

```
cd ./Progetti      # dalla directory corrente entra in  
                   # Progetti
```


Esercizio: completate la tabella

Pathname assoluto	Directory corrente	Pathname relativo
/home/ciccioformaggio/Documents/document.odt	/home/ciccioformaggio/Documents	document.odt “oppure” ./document.odt
		home/ciccioformaggio/Documents/document.odt
	/home/ciccioformaggio/Images	
		../../document.odt
	/usr	
/home/ciccioformaggio/Documents	/home/ciccioformaggio/Documents	.
		home/ciccioformaggio/Documents
		..
	/home/ciccioformaggio/Images/sea	
		../home/ciccioformaggio/Documents

Operazioni sui file

- Creazione:

`touch <nome_file>`

- Visualizzazione contenuti:

`cat <file>` (tutto insieme)

`more <file> / less <file>` (una pagina alla volta)

- Spostamento o rinomina:

`mv <origine> <destinazione>`

- Copia:

`cp <origine> <destinazione>`

- Cancellazione:

`rm <file>`

(Attenzione: la shell di Linux NON CHIEDE CONFERMA prima di cancellare)

Operazioni con le directory

- Creazione:

```
mkdir <nome_directory>
```

- Spostamento o rinomina:

```
mv <origine> <destinazione>
```

- Creazione di una copia:

```
cp -r <origine> <destinazione>
```

- Cancellazione/Eliminazione:

- `rmdir <nome_directory>` → solo se vuota
- `rm -r <nome_directory>` → elimina **ricorsivamente** il contenuto della directory e delle sue sottodirectory (Consiglio: Usare `rm -r` con cautela per evitare la perdita di dati.)

Wildcard

- Quando si specificano nomi di file o directory, è possibile usare caratteri jolly (wildcard) per indicare più elementi contemporaneamente:
 - * → qualsiasi sequenza di caratteri
 - ? → un singolo carattere
 - [] → uno qualsiasi dei caratteri inclusi tra le parentesi
- Esempi:
 - *.txt → tutti i file con estensione .txt
 - file?.txt → file come file1.txt, fileA.txt, ecc.
 - file[12].txt → file1.txt o file2.txt
- Le wildcard sono molto utili per operazioni di ricerca, copia o cancellazione di più file.

Permessi su file e directory

- Gli utenti possono assegnare tre tipi di **permessi di accesso**:
 - **Read (R)** → permette di leggere il contenuto di un file o di una directory
 - **Write (W)** → permette di modificare il contenuto di un file o di una directory
 - **Execute (X)** → permette di eseguire un file o di accedere a una directory
- **Tre livelli di permessi**:
 - **Owner (u)** → utente che crea il file o la directory
 - **Group (g)** → utenti dello stesso gruppo del proprietario
 - **Others (o)** → tutti gli altri utenti
- **Rappresentazione**:
 - I permessi sono indicati con un terzetto di caratteri per ciascun livello, ad esempio:
`rwX r-X r--`
dove il primo terzetto è per l'owner, il secondo per il group e il terzo per gli others.
- Combinando questi permessi si può controllare con precisione chi può leggere, modificare o eseguire un file/directory.

Ricerche nel file system

- `find` è un comando che permette di eseguire **ricerche ricorsive** a partire da una directory specificata.
- Caratteristiche principali:
 - Ricerca di file e directory in modo ricorsivo
 - Possibilità di usare nomi parziali o pattern
 - Filtraggio avanzato, ad esempio per:
 - Date di creazione o modifica
 - Tipologia di file
 - Dimensione
- `find` è molto potente e flessibile, ma richiede attenzione alla sintassi per ottenere i risultati desiderati.

find

- Ricerca per **nome**

```
find /home -name "file.txt"
```

→ cerca `file.txt` nella directory `/home` e sottodirectory

- Ricerca per **estensione**

```
find . -name "*.txt"
```

→ cerca tutti i file `.txt` nella directory corrente

- Ricerca per **tipo**

```
find /etc -type d
```

→ elenca tutte le directory sotto `/etc`

- Ricerca per **dimensione**

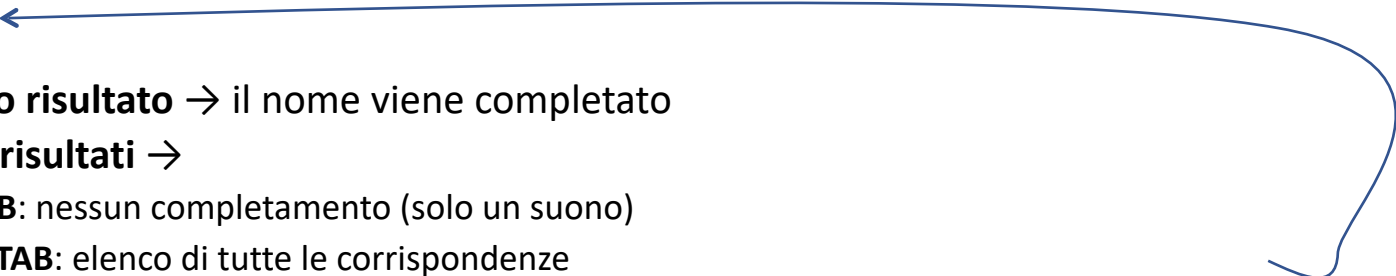
```
find . -size +1M
```

→ cerca file più grandi di 1 MB

Redirezione da/a file

- E' possibile usare file come input o output dei comandi.
- Redirezione dell'output (**stdout**):
 - `>` → scrive l'output in un file (sovrascrive il contenuto)
`ls > elenco.txt`
 - `>>` → aggiunge l'output in fondo a un file esistente
`ls >> elenco.txt`
- Redirezione dell'input (**stdin**):
 - `<` → legge l'input da un file invece che da tastiera
`sort < elenco.txt`
- La redirezione è utile per salvare risultati, concatenare comandi o leggere dati da file.

Completamento automatico

- La shell permette di scrivere più velocemente nomi di file e comandi grazie al tasto **TAB**.
- Funzionamento base:
 - Scrivere le prime lettere di un file/directory/comando
 - Premere **TAB** per completare automaticamente
- Casi possibili: 
 - Se esiste **un solo risultato** → il nome viene completato
 - Se esistono **più risultati** →
 - al primo **TAB**: nessun completamento (solo un suono)
 - al secondo **TAB**: elenco di tutte le corrispondenze
 - continuare a scrivere fino a **disambiguare** (almeno parzialmente), poi premere di nuovo **TAB**
- Valido per:
 - Nomi di **file** e **directory**
 - Nomi di **comandi** della shell
- Usare il completamento automatico riduce errori di digitazione e fa risparmiare tempo.

Il manuale

- Il comando `man` permette di visualizzare la **manual page** di un comando, ovvero una descrizione dettagliata delle sue funzionalità e opzioni.
- Sintassi base:
`man <comando>`
- Esempi pratici:
 - Visualizzare il manuale del comando `ls`:
`man ls`
 - Consultare le opzioni del comando `find`:
`man find`
- Navigazione nel manuale:
 - Freccia su/giù → scorrere pagina per pagina
 - Space → pagina successiva
 - q → uscire dal manuale
- `man` è uno strumento fondamentale per capire opzioni e comportamento di qualsiasi comando della shell.